AGLC CRM Local Host Deployment

Complete Terminal Commands & Scripts

Date: March 13, 2026
Environment: Local Development
Status: Ready to Deploy

SECTION 1: PREREQUISITES & INITIAL SETUP

1.1 Install Required Tools

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install Node.js 22 (if not already installed)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Verify Node.js installation
node --version  # Should be v22.13.0 or higher
npm --version

# Install pnpm globally
sudo npm install -g pnpm@10.4.1

# Verify pnpm installation
pnpm --version  # Should be 10.4.1 or higher

# Install Docker (if not already installed)
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker

# Verify Docker installation
docker --version
docker-compose --version

# Install PostgreSQL client (for local development)
sudo apt install -y postgresql-client

# Install Git (if not already installed)
sudo apt install -y git
```

SECTION 2: GITHUB REPOSITORY SETUP

2.1 Create GitHub Organization & Repositories

```
# Login to GitHub CLI (if not already logged in)
gh auth login

# Create GitHub organization (optional - if you want a separate org)
# gh org create aglc --display-name "AGLC Law Firm"

# Create main CRM repository
gh repo create aglc-crm \
  --private \
  --description "AGLC Law Firm CRM - Main Application" \
  --homepage "https://aglc-crm.local" \
  --gitignore Node \
  --license MIT

# Create AI agents repository
gh repo create aglc-crm-ai-agents \
  --private \
  --description "AGLC CRM - AI Agents & Orchestration" \
  --gitignore Node \
  --license MIT

# Create documentation repository
gh repo create aglc-crm-docs \
  --private \
  --description "AGLC CRM - Documentation & Knowledge Base" \
  --gitignore Node \
  --license MIT

# Clone repositories to local machine
mkdir -p ~/projects/aglc
cd ~/projects/aglc

gh repo clone aglc-crm
gh repo clone aglc-crm-ai-agents
gh repo clone aglc-crm-docs

# Verify repositories
ls -la
```

SECTION 3: PROJECT INITIALIZATION

3.1 Initialize Main CRM Repository

```
cd ~/projects/aglc/aglc-crm

# Initialize git repository
git init
git config user.email "your-email@aglc.com"
git config user.name "Your Name"

# Create main branch
git checkout -b main

# Copy project files from template (if using Manus template)
# This assumes you've already initialized the project with
webdev_init_project
# If not, initialize a new Node.js project:

# Initialize package.json
pnpm init -y

# Install core dependencies
pnpm install \
  @trpc/client \
  @trpc/react-query \
  @trpc/server \
  react \
  react-dom \
  express \
  drizzle-orm \
  mysql2 \
  typescript \
  vite \
  tailwindcss \
  @tailwindcss/vite

# Install dev dependencies
pnpm add -D \
  @types/node \
  @types/react \
  @types/react-dom \
  @types/express \
  typescript \
```

```
  vitest \
  tsx \
  drizzle-kit \
  esbuild

# Create directory structure
mkdir -p client/src/{pages,components,contexts,hooks,lib}
mkdir -p server/{routers,services,integrations,middleware,_core}
mkdir -p drizzle/{migrations}
mkdir -p tests/{unit,integration,e2e}
mkdir -p docs
mkdir -p .github/workflows

# Create initial configuration files
touch .env.example
touch .env.local
touch tsconfig.json
touch vitest.config.ts
touch docker-compose.yml
touch Dockerfile

# Create .gitignore
cat > .gitignore << 'EOF'
# Dependencies
node_modules/
pnpm-lock.yaml
package-lock.json
yarn.lock

# Environment
.env
.env.local
.env.*.local

# Build outputs
dist/
build/
.next/
out/

# IDE
.vscode/
.idea/
*.swp
*.swo
*~
```

```
.DS_Store

# Testing
coverage/
.nyc_output/

# Logs
logs/
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# OS
.DS_Store
Thumbs.db

# Database
*.db
*.sqlite
EOF

# Create initial commit
git add .
git commit -m "Initial project setup"

# Add remote and push
git remote add origin https://github.com/your-username/aglc-crm.git
git branch -M main
git push -u origin main
```

3.2 Initialize AI Agents Repository

```
cd ~/projects/aglc/aglc-crm-ai-agents

# Initialize git
git init
git config user.email "your-email@aglc.com"
git config user.name "Your Name"

# Create directory structure
mkdir -p agents/{briefing-wizard,legal-assistant,translator,business-card-
scanner}/src
mkdir -p orchestration/src
mkdir -p shared/src
mkdir -p docs
mkdir -p .github/workflows

# Create package.json for root
pnpm init -y

# Install dependencies
pnpm install \
  openai \
  zod \
  typescript

# Create .gitignore
cat > .gitignore << 'EOF'
node_modules/
pnpm-lock.yaml
dist/
build/
.env
.env.local
coverage/
logs/
*.log
.DS_Store
.vscode/
.idea/
EOF

# Create initial commit
git add .
git commit -m "Initial AI agents project setup"
```

```
# Add remote and push
git remote add origin https://github.com/your-username/aglc-crm-ai-agents.git
git branch -M main
git push -u origin main
```

3.3 Initialize Documentation Repository

```
cd ~/projects/aglc/aglc-crm-docs

# Initialize git
git init
git config user.email "your-email@aglc.com"
git config user.name "Your Name"

# Create directory structure
mkdir -p guides
mkdir -p workflows
mkdir -p specifications
mkdir -p team
mkdir -p operations
mkdir -p images
mkdir -p .github/workflows

# Create README
cat > README.md << 'EOF'
# AGLC CRM Documentation

Complete documentation for the AGLC Law Firm CRM system.

## Quick Links

- [Getting Started](./guides/GETTING_STARTED.md)
- [Architecture](./guides/ARCHITECTURE.md)
- [API Documentation](./guides/API.md)
- [Module Specifications](./specifications/MODULE_SPECIFICATIONS.md)

## Documentation Structure

- **guides/** - Comprehensive guides and tutorials
- **workflows/** - Business process workflows
- **specifications/** - Technical specifications
- **team/** - Team guidelines and standards
- **operations/** - Operations and maintenance procedures
- **images/** - Documentation images and diagrams

EOF

# Create .gitignore
cat > .gitignore << 'EOF'
.DS_Store
```

```
.vscode/
.idea/
*.swp
*.swo
*~
EOF

# Create initial commit
git add .
git commit -m "Initial documentation repository setup"

# Add remote and push
git remote add origin https://github.com/your-username/aglc-crm-docs.git
git branch -M main
git push -u origin main
```

SECTION 4: DATABASE SETUP

4.1 PostgreSQL Setup with Docker

```
# Create docker-compose.yml for local development
cat > ~/projects/aglc/aglc-crm/docker-compose.yml << 'EOF'
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: aglc-crm-postgres
    environment:
      POSTGRES_USER: aglc_user
      POSTGRES_PASSWORD: aglc_password_dev
      POSTGRES_DB: aglc_crm_dev
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U aglc_user"]
      interval: 10s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: aglc-crm-pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@aglc.com
      PGADMIN_DEFAULT_PASSWORD: admin_password_dev
    ports:
      - "5050:80"
    depends_on:
      - postgres

volumes:
  postgres_data:

networks:
  default:
    name: aglc-crm-network
EOF
```

```
# Start PostgreSQL and pgAdmin
cd ~/projects/aglc/aglc-crm
docker-compose up -d

# Verify containers are running
docker-compose ps

# Check PostgreSQL logs
docker-compose logs postgres

# Access pgAdmin at http://localhost:5050
# Email: admin@aglc.com
# Password: admin_password_dev
```

4.2 Create Database Schema

```
cd ~/projects/aglc/aglc-crm

# Create .env.local with database connection
cat > .env.local << 'EOF'
# Database
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_

# OAuth (Manus)
VITE_APP_ID=your_app_id_here
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im

# External Services
DOCUSIGN_API_KEY=your_docusign_key
DOCUSIGN_ACCOUNT_ID=your_account_id
PLATFORM_API_KEY=your_platform_key
CALENDLY_API_KEY=your_calendly_key
SENDGRID_API_KEY=your_sendgrid_key
TWILIO_ACCOUNT_SID=your_twilio_sid
TWILIO_AUTH_TOKEN=your_twilio_token
LLM_API_KEY=your_llm_key

# Application
JWT_SECRET=your_jwt_secret_here_min_32_chars
NODE_ENV=development
PORT=3000
EOF

# Install dependencies
pnpm install

# Run database migrations
pnpm db:push

# Seed database (optional)
pnpm db:seed
```

SECTION 5: DEVELOPMENT SERVER SETUP

5.1 Start Development Server

```
cd ~/projects/aglc/aglc-crm

# Install all dependencies
pnpm install

# Start development server
pnpm dev

# The server will start on http://localhost:3000
# Frontend: http://localhost:5173 (Vite dev server)
# Backend API: http://localhost:3000/api/trpc

# In another terminal, watch for tests
pnpm test --watch

# In another terminal, watch for type errors
pnpm check --watch
```

5.2 Verify Development Setup

```
# Check if frontend is running
curl http://localhost:5173

# Check if backend is running
curl http://localhost:3000/api/trpc

# Check database connection
psql -U aglc_user -d aglc_crm_dev -h localhost -c "SELECT version();"

# Check pgAdmin
# Open browser: http://localhost:5050
```

SECTION 6: GITHUB ACTIONS SETUP

6.1 Create CI/CD Workflows

```
cd ~/projects/aglc/aglc-crm

# Create GitHub workflows directory
mkdir -p .github/workflows

# Create CI workflow
cat > .github/workflows/ci.yml << 'EOF'
name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: '22'
          cache: 'pnpm'
```

```
      - run: pnpm install
      - run: pnpm run check
      - run: pnpm run format --check
      - run: pnpm run test
      - run: pnpm run build

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/coverage-final.json
EOF

# Create security workflow
cat > .github/workflows/security.yml << 'EOF'
name: Security Scanning

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]
  schedule:
    - cron: '0 0 * * 0'

jobs:
  security:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: '22'
          cache: 'pnpm'

      - run: pnpm install
      - run: pnpm audit --audit-level=moderate
EOF

# Commit workflows
git add .github/workflows/
git commit -m "Add GitHub Actions CI/CD workflows"
git push origin main
```

SECTION 7: LOCAL DEPLOYMENT SCRIPTS

7.1 Create Deployment Script

```
# Create deployment script
cat > ~/projects/aglc/deploy-local.sh << 'EOF'
#!/bin/bash

set -e

echo "🚀 AGLC CRM Local Deployment Script"
echo "===================================="

# Colors for output
GREEN='\033[0;32m'
BLUE='\033[0;34m'
RED='\033[0;31m'
NC='\033[0m' # No Color

# Project directory
PROJECT_DIR="$HOME/projects/aglc/aglc-crm"

echo -e "${BLUE}Step 1: Checking prerequisites...${NC}"
if ! command -v node &> /dev/null; then
    echo -e "${RED}Node.js is not installed${NC}"
    exit 1
fi
echo -e "${GREEN}✓ Node.js $(node --version) is installed${NC}"

if ! command -v pnpm &> /dev/null; then
    echo -e "${RED}pnpm is not installed${NC}"
    exit 1
fi
echo -e "${GREEN}✓ pnpm $(pnpm --version) is installed${NC}"

if ! command -v docker &> /dev/null; then
    echo -e "${RED}Docker is not installed${NC}"
    exit 1
fi
echo -e "${GREEN}✓ Docker is installed${NC}"

echo -e "${BLUE}Step 2: Starting PostgreSQL with Docker...${NC}"
cd "$PROJECT_DIR"
docker-compose up -d
```

```
echo -e "${GREEN}✓ PostgreSQL and pgAdmin are running${NC}"

echo -e "${BLUE}Step 3: Waiting for PostgreSQL to be ready...${NC}"
sleep 5

echo -e "${BLUE}Step 4: Installing dependencies...${NC}"
pnpm install
echo -e "${GREEN}✓ Dependencies installed${NC}"

echo -e "${BLUE}Step 5: Running database migrations...${NC}"
pnpm db:push
echo -e "${GREEN}✓ Database migrations completed${NC}"

echo -e "${BLUE}Step 6: Building project...${NC}"
pnpm build
echo -e "${GREEN}✓ Project built successfully${NC}"

echo ""
echo -e "${GREEN}===================================="
echo "✓ Deployment Complete!"
echo "====================================${NC}"
echo ""
echo "📍 Access Points:"
echo "  - Frontend: http://localhost:5173"
echo "  - Backend API: http://localhost:3000/api/trpc"
echo "  - Database: localhost:5432"
echo "  - pgAdmin: http://localhost:5050"
echo ""
echo "📋 Next Steps:"
echo "  1. Start development server: cd $PROJECT_DIR && pnpm dev"
echo "  2. Open http://localhost:5173 in your browser"
echo "  3. Login with your credentials"
echo ""
echo "🛑 To stop services:"
echo "  docker-compose down"
echo ""

EOF

# Make script executable
chmod +x ~/projects/aglc/deploy-local.sh

# Run deployment script
~/projects/aglc/deploy-local.sh
```

7.2 Create Start/Stop Scripts

```
# Create start script
cat > ~/projects/aglc/start.sh << 'EOF'
#!/bin/bash

PROJECT_DIR="$HOME/projects/aglc/aglc-crm"

echo "🚀 Starting AGLC CRM..."
cd "$PROJECT_DIR"

# Start Docker containers
echo "Starting Docker containers..."
docker-compose up -d

# Wait for services to be ready
echo "Waiting for services to be ready..."
sleep 5

# Start development server
echo "Starting development server..."
pnpm dev

EOF

chmod +x ~/projects/aglc/start.sh

# Create stop script
cat > ~/projects/aglc/stop.sh << 'EOF'
#!/bin/bash

PROJECT_DIR="$HOME/projects/aglc/aglc-crm"

echo "🛑 Stopping AGLC CRM..."
cd "$PROJECT_DIR"

# Stop development server (Ctrl+C in terminal)
echo "Stopping development server..."
# Note: You'll need to press Ctrl+C in the terminal running pnpm dev

# Stop Docker containers
echo "Stopping Docker containers..."
docker-compose down

echo "✓ AGLC CRM stopped"
```

```
EOF

chmod +x ~/projects/aglc/stop.sh

# Create reset script
cat > ~/projects/aglc/reset.sh << 'EOF'
#!/bin/bash

PROJECT_DIR="$HOME/projects/aglc/aglc-crm"

echo "⚠️  Resetting AGLC CRM (This will delete all data)..."
read -p "Are you sure? (y/n) " -n 1 -r
echo
if [[ $REPLY =~ ^[Yy]$ ]]; then
    cd "$PROJECT_DIR"

    # Stop and remove containers
    docker-compose down -v

    # Remove node_modules and reinstall
    rm -rf node_modules pnpm-lock.yaml
    pnpm install

    # Restart containers
    docker-compose up -d
    sleep 5

    # Run migrations
    pnpm db:push

    echo "✓ Reset complete"
else
    echo "Reset cancelled"
fi

EOF

chmod +x ~/projects/aglc/reset.sh
```

SECTION 8: QUICK START COMMANDS

8.1 Complete Quick Start

```
# 1. Clone all repositories
mkdir -p ~/projects/aglc
cd ~/projects/aglc

gh repo clone aglc-crm
gh repo clone aglc-crm-ai-agents
gh repo clone aglc-crm-docs

# 2. Setup main CRM project
cd aglc-crm

# 3. Create environment file
cat > .env.local << 'EOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
EOF

# 4. Start PostgreSQL
docker-compose up -d

# 5. Install dependencies
pnpm install

# 6. Run migrations
pnpm db:push

# 7. Start development server
pnpm dev

# 8. In another terminal, start tests
pnpm test --watch

# 9. Open browser
# Frontend: http://localhost:5173
```

```
# Backend: http://localhost:3000
# pgAdmin: http://localhost:5050
```

SECTION 9: TROUBLESHOOTING

9.1 Common Issues & Solutions

```
# Issue: PostgreSQL connection refused
# Solution: Check if container is running
docker-compose ps

# Restart containers
docker-compose restart postgres

# Issue: Port already in use
# Solution: Kill process using port 3000
lsof -ti:3000 | xargs kill -9

# Kill process using port 5432
lsof -ti:5432 | xargs kill -9

# Issue: pnpm install fails
# Solution: Clear cache and reinstall
pnpm store prune
rm -rf node_modules pnpm-lock.yaml
pnpm install

# Issue: Database migration fails
# Solution: Check database connection
psql -U aglc_user -d aglc_crm_dev -h localhost -c "SELECT version();"

# Reset database
docker-compose down -v
docker-compose up -d
sleep 5
pnpm db:push

# Issue: TypeScript errors
# Solution: Run type check
pnpm check

# Issue: Tests failing
# Solution: Run tests with verbose output
pnpm test --reporter=verbose
```

9.2 Useful Commands

```
# View Docker logs
docker-compose logs -f postgres

# Access PostgreSQL CLI
docker-compose exec postgres psql -U aglc_user -d aglc_crm_dev

# View running processes
ps aux | grep node

# Check port usage
netstat -tulpn | grep LISTEN

# View environment variables
env | grep DATABASE_URL

# Test API endpoint
curl http://localhost:3000/api/trpc

# Check frontend build
pnpm build

# Run linter
pnpm lint

# Format code
pnpm format
```

SECTION 10: DEPLOYMENT CHECKLIST

```
# Pre-deployment checklist
echo "Pre-Deployment Checklist:"
echo "☐ Node.js 22.13.0 or higher installed"
echo "☐ pnpm 10.4.1 or higher installed"
echo "☐ Docker and Docker Compose installed"
echo "☐ PostgreSQL 15 running in Docker"
echo "☐ .env.local file created with all required variables"
echo "☐ Dependencies installed (pnpm install)"
echo "☐ Database migrations completed (pnpm db:push)"
echo "☐ Project builds successfully (pnpm build)"
echo "☐ Tests pass (pnpm test)"
echo "☐ No TypeScript errors (pnpm check)"
echo "☐ Code is formatted (pnpm format)"
echo ""
echo "Deployment checklist complete!"
```

FINAL NOTES

Important Security Notes:

- Never commit .env.local to Git
- Change default passwords in production
- Use strong JWT_SECRET (minimum 32 characters)
- Enable HTTPS in production
- Implement proper authentication
- Use environment variables for all secrets

Performance Tips:

- Use pnpm for faster installations
- Enable Docker BuildKit for faster builds
- Use development mode for local testing
- Monitor database performance

- Implement caching strategies

Next Steps:

1. Run the deployment script
2. Verify all services are running
3. Access the application at http://localhost:5173
4. Begin development on Phase 1 features
5. Commit changes to GitHub

Status: READY FOR LOCAL DEPLOYMENT
Timeline: 15-20 minutes for complete setup
Support: Refer to troubleshooting section for common issues